

ল্যাপটপ-বিহীন সম্পূর্ণ মোবাইল-ভিত্তিক এআই অ্যাপ্লিকেশন ইঞ্জিন এবং এপিকে (APK) জেনারেশন: একটি এলনাইন মেটা-আর্কিটেকচার (L9_META_DESIGNER) বিশ্লেষণ

আধুনিক সফটওয়্যার ইঞ্জিনিয়ারিং এবং ক্লাউড-নেটিভ টুলসের বিবর্তনের ফলে প্রথাগত ডেস্কটপ-ভিত্তিক প্রোগ্রামিংয়ের প্রয়োজনীয়তা ক্রমশ হ্রাস পাচ্ছে। প্রথাগতভাবে, অ্যান্ড্রয়েড অ্যাপ্লিকেশন (APK) তৈরি করার জন্য একটি শক্তিশালী কম্পিউটার, অ্যান্ড্রয়েড স্টুডিও (Android Studio), জাভা ডেভেলপমেন্ট কিট (JDK), এবং একাধিক গিগাবাইটের অ্যান্ড্রয়েড এসডিকে (SDK) প্রয়োজন হয়। তবে, কোনো কম্পিউটার বা ল্যাপটপ ব্যবহার ছাড়াই, সম্পূর্ণ একটি মোবাইল ডিভাইস এবং কৃত্রিম বুদ্ধিমত্তা (AI) ব্যবহার করে একটি স্বয়ংক্রিয় 'অ্যান্ড্রয়েড অ্যাপ্লিকেশন ইঞ্জিন' তৈরি করা প্রযুক্তিগতভাবে সম্ভব। এই ধরনের একটি ইঞ্জিন—যা ওয়েব অ্যাপের জিপ (ZIP) ফাইল বা ক্লিপট থেকে স্বয়ংক্রিয়ভাবে নেটিভ অ্যান্ড্রয়েড কোড জেনারেট করবে এবং সরাসরি এপিকে (APK) হিসেবে এক্সপোর্ট করবে—তার জন্য ক্লাউড এআই সক্ষমতা এবং লোকাল মোবাইল কমান্ড-লাইন ইন্টারফেসের একটি সুনির্দিষ্ট ও সূক্ষ্ম সমন্বয় প্রয়োজন। এই প্রতিবেদনে গুগল এআই স্টুডিও (Google AI Studio)-এর বর্তমান সক্ষমতা, এর ফ্রি টায়ার লিমিটেশন, অন-ডিভাইস কম্পাইলেশন (On-device compilation) প্রযুক্তির খুঁটিনাটি এবং প্রস্তাবিত ইঞ্জিনটি নির্মাণের জন্য প্রয়োজনীয় ম্যাটেরিয়াল ও সেগুলোর সেটআপের একটি বিস্তারিত কারিগরি গাইডলাইন প্রদান করা হয়েছে।

গুগল এআই স্টুডিও এবং অ্যাপ বিল্ডারের বর্তমান সক্ষমতা যাচাই

গুগল এআই স্টুডিও মূলত জেমিনাই (Gemini) মডেলগুলো পরীক্ষা করার একটি সাধারণ প্ল্যাটফর্ম থেকে একটি পূর্ণাঙ্গ ফুল-স্ট্যাক অ্যাপ বিল্ডিং প্ল্যাটফর্মে রূপান্তরিত হয়েছে। ২০২৬ সালের মে মাসে অনুষ্ঠিত গুগল আই/ও (Google I/O) সম্মেলনের আপডেট অনুযায়ী, এআই স্টুডিওর "বিল্ড মোড" (Build Mode) ব্যবহার করে প্রম্পট বা ন্যাচারাল ল্যাঙ্গুয়েজের মাধ্যমে সরাসরি অ্যাপ্লিকেশন তৈরি করা যায়, যাকে প্রযুক্তি দুনিয়ায় "ভাইব কোডিং" (Vibe coding) বলা হচ্ছে।

এআই স্টুডিওর নেটিভ সক্ষমতা

এআই স্টুডিও নিজে থেকে যে কাজগুলো করতে পারে, তার একটি বিস্তারিত মূল্যায়ন নিচে দেওয়া হলো। এই সক্ষমতাগুলোকে অগ্রাধিকার দিয়েই ইঞ্জিনের পরবর্তী আর্কিটেকচার ডিজাইন করা হয়েছে। প্রথমত, এআই স্টুডিও বর্তমানে শুধুমাত্র ওয়েব অ্যাপ নয়, বরং নেটিভ অ্যান্ড্রয়েড অ্যাপ তৈরি করতে সক্ষম। ব্যবহারকারী একটি সাধারণ প্রম্পট প্রদান করলে এর পেছনের 'অ্যান্টিগ্র্যাভিটি এজেন্ট' (Antigravity Agent) স্বয়ংক্রিয়ভাবে কোটলিন (Kotlin) এবং লেটেস্ট অ্যাপ্লিকেশন ডেভেলপমেন্ট প্যাটার্ন 'জেটপ্যাক কম্পোজ' (Jetpack Compose) ব্যবহার করে প্রডাকশন-মানের কোড তৈরি করে। এই প্রক্রিয়ায় কোনো থার্ড-পার্টি লাইব্রেরি বা এক্সটার্নাল এডিটর ছাড়াই সরাসরি ব্রাউজারের ভেতর একটি পূর্ণাঙ্গ সিঙ্গেল-অ্যাক্টিভিটি আর্কিটেকচার (Single-activity architecture) দাঁড় করানো হয়।

দ্বিতীয়ত, কোড জেনারেট হওয়ার সাথে সাথে এআই স্টুডিও ব্রাউজারের ভেতরেই একটি পিক্সেল-সদৃশ এমুলেটর রান করে, যেখানে ট্যাপ, স্ক্রল এবং অ্যাপের ইউজার ইন্টারফেস (UI) রিয়েল-টাইমে পরীক্ষা করা যায়। এই এমুলেটরটি ফিজিক্যাল ডিভাইসের মতো কাজ করলেও এতে ক্লটুথ, এনএফসি (NFC) বা হার্ডওয়্যার ক্যামেরা সাপোর্ট থাকে না, তবে সফটওয়্যার ডেভেলপমেন্টের প্রাথমিক ধাপগুলো যাচাই করার জন্য এটি অত্যন্ত কার্যকর।

তৃতীয়ত, ডেটাবেস এবং ব্যাকএন্ড ম্যানেজমেন্টের ক্ষেত্রে এআই স্টুডিওর ফায়ারবেস ইন্টিগ্রেশন (Firebase Integration) একটি বড় ভূমিকা পালন করে। ওয়েব অ্যাপ্লিকেশনের ক্ষেত্রে এটি স্বয়ংক্রিয়ভাবে ফায়ারবেস ফায়ারস্টোর (Firestore) এবং ফায়ারবেস অথেনটিকেশন (Authentication) যুক্ত করতে পারে এবং প্রয়োজনীয় ডেটাবেস রুলস

সেটআপ করে দেয়। যদিও অ্যান্ড্রয়েড অ্যাপ্লিকেশনের জন্য সরাসরি এআই স্টুডিওর বিল্ড মোড থেকে ফায়ারবেস অথেনটিকেশন ইন্টিগ্রেশন ২০২৬ সাল পর্যন্ত সীমাবদ্ধ, তবে এটি প্রয়োজনীয় এপিআই (API) কল করার লজিক এবং ইউআই জেনারেট করতে সক্ষম।

চতুর্থত, সরাসরি ডিভাইসে ইন্সটলেশন করার জন্য গুগল এআই স্টুডিও অ্যান্ড্রয়েড ডিবাগ ব্রিজ (ADB) এবং WebUSB প্রযুক্তি ব্যবহার করে। পিসির সাথে ইউএসবি ক্যাবল যুক্ত থাকলে ব্রাউজার থেকেই একটি ফিজিক্যাল অ্যান্ড্রয়েড ডিভাইসে অ্যাপ ইন্সটল করা যায়। সবশেষে, একটি গুগল প্লে ডেভেলপার অ্যাকাউন্ট (Google Play Developer account) কানেক্ট করা থাকলে, এক ক্লিকেই অ্যাপটি ইন্টারনাল টেস্টিং ট্র্যাকে (Internal Test Track) পাবলিশ করা যায়, যা ১০০ জন টেস্টার পর্যন্ত সীমাবদ্ধ থাকে।

ফ্রি টিয়ার ক্যাপাবিলিটি (Free Tier Capabilities)

গুগল এআই স্টুডিও এবং জেমিনাই এপিআই (Gemini API) প্রোটোটাইপিংয়ের জন্য একটি শক্তিশালী ফ্রি টিয়ার প্রদান করে, যা ডেভেলপারদের কোনো ক্রেডিট কার্ড ছাড়াই এন্টারপ্রাইজ-গ্রেড ইনফ্রাস্ট্রাকচার ব্যবহার করার সুযোগ দেয়। ব্যবহারকারীরা গুগল ক্লাউড রান (Cloud Run)-এ সর্বোচ্চ দুটি ওয়েব অ্যাপ্লিকেশন সম্পূর্ণ বিনামূল্যে ডেপ্লয় করতে পারেন। এছাড়া, জেমিনাই ডেভেলপার এপিআই (Gemini Developer API) বিনামূল্যে ব্যবহার করা যায়, যা সার্ভার-সাইড স্ক্রিপ্ট বা অ্যাপ ইঞ্জিনের ব্যাকএন্ড থেকে কল করা সম্ভব। তবে ফ্রি টিয়ারে নির্দিষ্ট রেট লিমিট (Rate limits) থাকে এবং মডেল ট্রেনিংয়ের জন্য গুগল ডেটা সংগ্রহ করতে পারে।

সরাসরি এপিকে (APK) এক্সপোর্টের ক্ষেত্রে এআই স্টুডিওর সীমাবদ্ধতা

এআই স্টুডিওর অ্যাপ বিল্ডারের সবচেয়ে বড় সীমাবদ্ধতা হলো এটি সরাসরি কোনো কমপাইলড .apk বা প্রোডাকশন রেডি .aab ফাইল ডাউনলোডের অপশন প্রদান করে না। অ্যাপ্লিকেশনের সোর্স কোডটি একটি জিপ (ZIP) ফাইল হিসেবে ডাউনলোড করা যায়, অথবা সরাসরি কম্পিউটারের সাথে সংযুক্ত ডিভাইসে WebUSB-এর মাধ্যমে পুশ করা যায়। ডাউনলোড করা জিপ ফাইলটিতে অনেক সময় শুধুমাত্র সোর্স কোড থাকে এবং গ্র্যাডল (Gradle) বিল্ড কনফিগারেশনের কিছু অংশ অসম্পূর্ণ থাকতে পারে, যা পরবর্তীতে অ্যান্ড্রয়েড স্টুডিওতে ইম্পোর্ট করে ঠিক করতে হয়। যেহেতু মূল লক্ষ্য হলো সম্পূর্ণ মোবাইল ডিভাইসে ব্যবহার করে সরাসরি এপিকে (APK) ফাইল এক্সপোর্ট করা এবং কম্পিউটারের উপর নির্ভরশীলতা শূন্যতে নামিয়ে আনা, তাই এআই স্টুডিওর নিজস্ব ইউজার ইন্টারফেস (UI) দিয়ে এই ইঞ্জিন তৈরি সম্ভব নয়। এই শূন্যস্থান পূরণের জন্যই জেমিনাই এপিআই এবং লোকাল মোবাইল টার্মিনাল ব্যবহার করে একটি কাস্টম অ্যাপ্লিকেশন ইঞ্জিন ডিজাইন করা অপরিহার্য।

অনলাইন এক্সপোর্ট সলিউশন এবং ওয়েবসাইড থেকে এপিকে রূপান্তরের মেকানিজম

ইন্ডাস্ট্রিতে বর্তমানে ওয়েবসাইট বা ওয়েব অ্যাপ থেকে এপিকে ফাইল তৈরি করার বিভিন্ন থার্ড-পার্টি প্ল্যাটফর্ম রয়েছে, যেগুলোর মেকানিজম বিশ্লেষণ করলে প্রস্তাবিত ইঞ্জিনের আর্কিটেকচার বোঝা সহজ হয়। প্ল্যাটফর্মগুলোর মধ্যে RapidNative, AppsGeyser, Median, Andromo, Appilix এবং Fastshot উল্লেখযোগ্য।

RapidNative এবং Fastshot-এর মতো এআই-চালিত প্ল্যাটফর্মগুলো প্রম্পট থেকে সরাসরি রিঅ্যাক্ট নেটিভ (React Native) বা এক্সপো (Expo) প্রোজেক্ট তৈরি করে এবং ক্লাউড ইনফ্রাস্ট্রাকচার ব্যবহার করে এপিকে বিল্ড করে।

অন্যদিকে, Median, AppsGeyser বা Appilix মূলত একটি ওয়েবভিউ (WebView) র্যাপার তৈরি করে, যেখানে ব্যবহারকারীর দেওয়া ওয়েবসাইটের ইউআরএল (URL) বা এইচটিএমএল (HTML) জিপ ফাইলটি একটি নেটিভ অ্যাপের খোলসের ভেতর রেন্ডার করা হয়। এই প্ল্যাটফর্মগুলো ক্লাউডে গ্র্যাডল বিল্ড প্রসেস রান করে এবং ইউজারকে একটি চূড়ান্ত এপিকে ফাইল ডাউনলোড করতে দেয়।

তবে, এই সলিউশনগুলোর মূল সমস্যা হলো এগুলো থার্ড-পার্টি সার্ভারের ওপর নির্ভরশীল, সাবস্ক্রিপশন ফি দাবি করে এবং ব্যবহারকারীর কোডের ওপর সম্পূর্ণ নিয়ন্ত্রণ প্রদান করে না। প্রস্তাবিত ইঞ্জিনের লক্ষ্য হলো এই সম্পূর্ণ বিল্ড প্রসেসটি ক্লাউডের বদলে সরাসরি ব্যবহারকারীর মোবাইল ডিভাইসের লোকাল স্টোরেজে সম্পন্ন করা। এর জন্য ক্যাপাসিটর (Capacitor) বা নেটিভ গ্র্যাডল বিল্ড মেকানিজম ব্যবহার করে ওয়েব ক্লিপিং থেকে এপিকে রূপান্তর করার একটি

অন-ডিভাইস পাইপলাইন তৈরি করতে হবে।

প্রস্তাবিত এআই অ্যাপ্লিকেশন ইঞ্জিনের মেটা-আর্কিটেকচার এবং এপিআই স্কেপ

একটি স্বয়ংসম্পূর্ণ এআই-চালিত অ্যাপ্লিকেশন ইঞ্জিন তৈরি করতে হলে এটিকে লজিক্যালি তিনটি প্রধান স্তরে (Layers) বিভক্ত করতে হবে। এই আর্কিটেকচারটি ইনপুট থেকে শুরু করে আউটপুট পর্যন্ত ডেটা প্রবাহকে নিয়ন্ত্রণ করবে। প্রথম স্তর হলো 'ইনপুট ও পার্সিং লেয়ার' (Input & Parsing Layer)। এই স্তরে ইঞ্জিনটি ব্যবহারকারীর দেওয়া ওয়েব অ্যাপ্লিকেশনের জিপ ফাইল বা ক্লিপট (HTML, CSS, JS, React ইত্যাদি) ইনপুট হিসেবে গ্রহণ করবে। একটি লোকাল পাইথন (Python) বা নোড.জেএস (Node.js) স্ক্রিপ্ট এই জিপ ফাইলটি এক্সট্রাক্ট করে এর ভেতরের কম্পোনেন্টগুলো বিশ্লেষণ করবে।

দ্বিতীয় স্তর হলো 'এআই ট্রান্সলেশন এবং কোড জেনারেশন লেয়ার' (AI Translation and Code Generation Layer)। এই স্তরে সরাসরি গুগল জেমিনাই ডেভেলপার এপিআই (Gemini Developer API) ব্যবহার করা হবে। এপিআই-এর স্কেপ এমনভাবে ডিজাইন করতে হবে যেন এটি এক্সট্রাক্ট করা ওয়েব কোডগুলোকে প্রস্পট হিসেবে গ্রহণ করে এবং সেগুলোকে নেটিভ অ্যান্ড্রয়েড কোডে (Kotlin এবং Jetpack Compose) রূপান্তরিত করে। বিকল্প হিসেবে, যদি ওয়েব অ্যাপটিকে সরাসরি ওয়েবভিউ হিসেবে প্যাকেজ করতে হয়, তবে এআই ইঞ্জিনটি ক্যাপাসিটর (Capacitor) বা কর্ডোভা (Cordova) ফ্রেমওয়ার্কের জন্য প্রয়োজনীয় কনফিগারেশন ফাইল (যেমন capacitor.config.json বা build.gradle) তৈরি করে দেবে। এই পর্যায়ে এপিআই কল করার সময় সার্ভার-সাইড সিক্রেট ম্যানেজমেন্ট এবং রেট লিমিট অপ্টিমাইজেশনের দিকে লক্ষ্য রাখতে হবে, যা এআই স্টুডিওর ফ্রি টিয়ারে সমর্থিত।

তৃতীয় এবং সবচেয়ে গুরুত্বপূর্ণ স্তর হলো 'অন-ডিভাইস কম্পাইলেশন লেয়ার' (On-Device Compilation Layer)। এআই দ্বারা জেনারেট করা বা রূপান্তরিত কোডগুলোকে লোকাল মোবাইল কমান্ড-লাইন ইন্টারফেসে (Termux) গ্র্যাডল (Gradle) টুলচেইন ব্যবহার করে কম্পাইল করা হবে। এই প্রক্রিয়ায় সোর্স কোডটি প্রথমে aapt2 দ্বারা রিসোর্স কম্পাইলেশন, তারপর javac দ্বারা ক্লাস ফাইল তৈরি, d8 বা dx দ্বারা ডেক্স (DEX) ফাইলে রূপান্তর এবং সবশেষে apksigner দ্বারা সাইন হয়ে একটি চূড়ান্ত এপিকে (APK) ফাইলে পরিণত হবে।

বাধ্যতামূলকভাবে সংগ্রহযোগ্য ম্যাটেরিয়ালস (Required Materials)

ইঞ্জিনটি তৈরি এবং সফলভাবে রান করার জন্য কিছু নির্দিষ্ট হার্ডওয়্যার, সফটওয়্যার, এপিআই এবং স্ক্রিপ্ট আবশ্যিকভাবে সংগ্রহ করতে হবে। নিচে একটি কাঠামোগত তালিকার মাধ্যমে এই ম্যাটেরিয়ালগুলো বিস্তারিতভাবে উপস্থাপন করা হলো।

ক্যাটাগরি	উপাদান (Material)	বিবরণ এবং প্রয়োজনীয়তা	সোর্স / ধরন
হার্ডওয়্যার	অ্যান্ড্রয়েড স্মার্টফোন	ন্যূনতম Android 11 বা তার ওপরের ভার্সন। গ্র্যাডল (Gradle) বিন্ড প্রসেস অত্যন্ত মেমরি-ইন্টেনসিভ হওয়ার কারণে ন্যূনতম 6GB থেকে 8GB RAM সম্বলিত একটি শক্তিশালী প্রসেসরের ডিভাইস প্রয়োজন।	ফিজিক্যাল ডিভাইস
কোর এনভায়রনমেন্ট	Termux (F-Droid ভার্সন)	এটি একটি লিনাক্স এমুলেটর যা মোবাইলে কমান্ড-লাইন টুলস রান করতে দেয়। গুগল প্লে স্টোরের ভার্সনটি ডেপ্রিকেটেড (Deprecated) এবং আউটডেটেড, তাই শুধুমাত্র F-Droid বা GitHub থেকে লেটেস্ট ভার্সন	লোকাল অ্যাপ

ক্যাটাগরি	উপাদান (Material)	বিবরণ এবং প্রয়োজনীয়তা	সোর্স / ধরন
		(v0.118.0+) নামাতে হবে।	
এপিআই (API)	Gemini API Key	ওয়েব অ্যাপের ক্লিপিন্ট থেকে অ্যান্ড্রয়েড কোড জেনারেট করার জন্য এআই ইঞ্জিনের মূল মস্তিষ্ক হিসেবে কাজ করবে জেমিনাই। এটি Google AI Studio থেকে সংগ্রহ করতে হবে।	ক্লাউড সার্ভিস
কম্পাইলার টুলস	OpenJDK 21 (LTS)	অ্যান্ড্রয়েড অ্যাপ কম্পাইল করার জন্য জাভা রানটাইম (JRE) এবং ডেভেলপমেন্ট কিট (JDK)। এটি টারমাক্সের প্যাকেজ ম্যানেজার ব্যবহার করে ইন্সটল করতে হবে।	বিল্ড টুলস
অ্যান্ড্রয়েড কোর	Android SDK & CLI Tools	aapt2, d8, apksigner, zipalign-এর মতো কোর কমান্ড-লাইন টুলস যা সোর্স কোডকে এপিকে প্যাকেজিং এবং সাইনিংয়ের জন্য আবশ্যিক। এর সাথে প্ল্যাটফর্ম API 35 (Android 15) প্রয়োজন হবে।	বিল্ড টুলস
অটোমেশন	Gradle v8.10.2	প্রোজেক্ট বিল্ড অটোমেশন সিস্টেম, যা সোর্স কোড থেকে স্বয়ংক্রিয়ভাবে ডিপেনডেন্সি ম্যানেজ করে এবং এপিকে ফাইল আউটপুট দেয়।	বিল্ড অটোমেশন
ইন্সটলার	AnyAPK	গুগল প্লে প্রোটেক্ট বা ভেওর ভেরিফিকেশন বাইপাস করে নিজস্ব তৈরি করা এপিকে ফাইল লোকালি ইন্সটল করার জন্য একটি ডেডিকেটেড টুল। এটি ওয়্যারলেস এডিবি (Wireless ADB) ব্যবহার করে কাজ করে এবং কোনো রুট (Root) পারমিশন চায় না।	টেস্টিং টুল
সেটআপ স্ক্রিপ্ট	Termux Build Scripts	মোবাইলে ম্যানুয়ালি SDK এবং JDK সেটআপ করা অত্যন্ত জটিল। তাই গিটহাবের ওপেন-সোর্স স্ক্রিপ্ট (যেমন: termux-build-apps বা termux-sdk-installer) সংগ্রহ করতে হবে যা এই প্রক্রিয়াকে স্বয়ংক্রিয় করবে।	অটোমেশন স্ক্রিপ্ট

ক্লিক-বাই-ক্লিক গাইড: সেটআপ এবং ইন্টিগ্রেশন

সম্পূর্ণ ইকোসিস্টেমটি একটি মোবাইল ডিভাইসে সেটআপ করা এবং প্রতিটি অংশকে একত্রে ইন্টিগ্রেট করে একটি সচল ইঞ্জিন তৈরি করার জন্য নিচের ক্লিক-বাই-ক্লিক গাইডটি অত্যন্ত সতর্কতার সাথে অনুসরণ করতে হবে। এই প্রক্রিয়াটি কয়েকটি ধাপে বিভক্ত।

ধাপ ১: জেমিনাই এপিআই (Gemini API) সংগ্রহ ও সংরক্ষণ

ইঞ্জিনের মূল ট্রান্সলেশন লেয়ার হিসেবে কাজ করবে গুগল জেমিনাই। এর এপিআই কী (API Key) সংগ্রহ করা প্রথম কাজ।

১. মোবাইল ব্রাউজার (Google Chrome বা Edge) থেকে সরাসরি aistudio.google.com/app/apikey ঠিকানায়ে প্রবেশ করতে হবে। ২. গুগল অ্যাকাউন্ট দিয়ে লগ-ইন করার পর ড্যাশবোর্ড থেকে "Create API Key" বাটনে ক্লিক করতে হবে। ৩. নতুন অথবা বিদ্যমান কোনো গুগল ক্লাউড প্রোজেক্ট (Google Cloud Project) সিলেক্ট করে কী (Key) জেনারেট করতে হবে। ৪. জেনারেট হওয়া দীর্ঘ আলফানিউমেরিক স্ট্রিংটি (API Key) কপি করে একটি সিকিউর নোটপ্যাড বা পাসওয়ার্ড ম্যানেজারে সংরক্ষণ করতে হবে। এই কী-টি কখনোই পাবলিক গিটহাব রিপোজিটরিতে বা ক্লায়েন্ট-সাইড সোর্স কোডে সরাসরি রাখা যাবে না, কারণ এটি লিক হলে সিকিউরিটি ঝুঁকি তৈরি হয়।

ধাপ ২: টারমাক্স (Termux) এবং লোকাল বিল্ড এনভায়রনমেন্ট কনফিগারেশন

এই পর্যায়ে মোবাইল ডিভাইসটিকে একটি পোর্টেবল কম্পাইলার বা বিল্ড সার্ভারে পরিণত করা হবে, যেখানে সম্পূর্ণ অফলাইনে কোড কম্পাইল হবে।

১. মোবাইল ব্রাউজার থেকে F-Droid ওয়েবসাইটে গিয়ে "Termux" লিখে সার্চ করতে হবে অথবা সরাসরি গিটহাব রিলিজ থেকে লেটেস্ট এপিকে (APK) ফাইলটি ডাউনলোড এবং ইন্সটল করতে হবে। গুগল প্লে স্টোরের ভার্সনটি কোনোভাবেই ব্যবহার করা যাবে না। ২. Termux ওপেন করে সর্বপ্রথম লোকাল স্টোরেজ অ্যাক্সেস পারমিশন দিতে হবে। টার্মিনালে নিচের কমান্ডটি টাইপ করে Enter চাপতে হবে:

```
termux-setup-storage
```

স্ক্রিনে একটি অ্যান্ড্রয়েড পপ-আপ এলে "Allow" বা "অনুমতি দিন" অপশন সিলেক্ট করতে হবে। ৩. টারমাক্সের ডিফল্ট প্যাকেজ লিস্ট আপডেট এবং আপগ্রেড করার জন্য কমান্ড দিতে হবে:

```
pkg update -y && pkg upgrade -y
```

৪. অ্যান্ড্রয়েড অ্যাপ বিল্ড করার জন্য প্রয়োজনীয় টুলস (Java, SDK, Gradle) ম্যানুয়ালি কনফিগার করা অত্যন্ত সময়সাপেক্ষ এবং ত্রুটিপূর্ণ হতে পারে। তাই গিটহাবের termux-build-apps নামক একটি প্রভেন অটোমেশন স্ক্রিপ্ট ব্যবহার করে এটি স্বয়ংক্রিয়ভাবে সেটআপ করা হবে। নিচের কমান্ডগুলো একে একে টার্মিনালে রান করতে হবে:

```
pkg install wget -y
wget -O ~/termux-setup.sh
https://raw.githubusercontent.com/illegalvoid/termux-build-apps/main/setup.sh
chmod +x ~/termux-setup.sh
bash ~/termux-setup.sh
```

এই স্ক্রিপ্টটি ব্যাকগ্রাউন্ডে কাজ করে স্বয়ংক্রিয়ভাবে OpenJDK 21, Android SDK (API 35), এবং Gradle 8.10.2 ডাউনলোড ও কনফিগার করবে। ইন্টারনেট স্পিডের ওপর ভিত্তি করে এই প্রসেসটি ১৫ থেকে ২০ মিনিট সময় নিতে পারে। ৫. সেটআপ সফলভাবে শেষ হলে এনভায়রনমেন্ট ভ্যারিয়েবল (Environment Variables) রিলোড করার জন্য নিচের কমান্ডটি দিতে হবে:

```
source ~/.bashrc
```

ধাপ ৩: **AAPT2** আর্কিটেকচার এরর ফিক্স করা (অত্যন্ত গুরুত্বপূর্ণ)

টারমাক্সে গ্র্যাডল (Gradle) দিয়ে নেটিভ বিল্ড করার সময় সবচেয়ে বড় এবং সাধারণ বাধা হলো aapt2 (Android Asset Packaging Tool 2) সম্পর্কিত সমস্যা। গ্র্যাডল ডিফল্টভাবে গুগল মাভেন (Google Maven) রিপোজিটরি থেকে লিনাক্সের x86_64 আর্কিটেকচারের জন্য aapt2 বাইনারি ডাউনলোড করে, যা অ্যান্ড্রয়েড মোবাইলের এআরএম (ARM/AArch64) প্রসেসরে রান করতে পারে না। এর ফলে "cannot execute: required file not found" বা রিসোর্স ওভারল্যাপিং এরর দেখা দেয়। এই সমস্যা সমাধানের জন্য টারমাক্সে আগে থেকে ইন্সটল হওয়া নেটিভ এআরএম-কম্প্যাটিবল aapt2 পাথটি গ্র্যাডলকে ম্যানুয়ালি চিনিয়ে দিতে হবে।

১. টারমাক্সে গ্র্যাডল প্রোপার্টিজ ফাইলটি তৈরি বা এডিট করতে কমান্ড দিতে হবে:

```
mkdir -p ~/.gradle
nano ~/.gradle/gradle.properties
```

২. ন্যানো (nano) টেক্সট এডিটরে নিচের লাইনটি হুবহু পেস্ট করতে হবে:

```
android.aapt2FromMavenOverride=/data/data/com.termux/files/usr/bin/aapt2
```

৩. Ctrl + O চেপে সেভ করে, Enter দিয়ে নিশ্চিত করে, এবং Ctrl + X চেপে এডিটর থেকে বের হতে হবে। এটি নিশ্চিত করবে যে গ্র্যাডল বিল্ডের সময় ভুল আর্কিটেকচারের টুল ব্যবহার করবে না।

ধাপ ৪: এআই ইঞ্জিন লজিক স্ক্রিপ্ট তৈরি (Python)

এই ধাপে একটি পাইথন (Python) স্ক্রিপ্ট তৈরি করতে হবে যা ব্যবহারকারীর দেওয়া ওয়েব জিপ (ZIP) বা ক্লিপার্ট ফাইলটি পার্স করবে, জেমিনাই এপিআই-এর সাথে যোগাযোগ করবে এবং প্রাপ্ত নেটিভ কোডগুলো টারমাক্সের বিল্ড ডিরেক্টরিতে সেভ করবে।

১. টারমাক্সে পাইথন এবং প্রয়োজনীয় মডিউল ইন্সটল করতে হবে:

```
pkg install python python-pip -y
pip install google-genai python-dotenv
```

২. ইঞ্জিনের কাজের সুবিধার জন্য একটি প্রধান ডিরেক্টরি তৈরি করতে হবে:

```
mkdir ~/ai-app-engine && cd ~/ai-app-engine
```

৩. একটি প্রমিত অ্যান্ড্রয়েড প্রোজেক্টের বেস স্ট্রাকচার (Base Structure) আগে থেকে গিটহাব থেকে ক্লোন করে রাখা সুবিধাজনক। এর ফলে এআই-কে পুরো প্রোজেক্ট স্ক্যাচ থেকে লিখতে হয় না, বরং এটি শুধুমাত্র লজিক এবং ইউআই ফাইলগুলো আপডেট করে।

```
git clone https://github.com/azmaveth/termux-android-hello.git
base-app
```

৪. একটি engine.py ফাইল তৈরি করে তাতে জেমিনাই এপিআই লজিক লিখতে হবে। লজিকের কোর মেকানিজম হবে এমন:

- স্ক্রিপ্টটি ইউজারের আপলোড করা ওয়েব জিপ ফাইলের (HTML/CSS/JS) কন্টেন্ট এক্সট্র্যাক্ট করে মেমোরিতে রিড করবে।
- জেমিনাইকে সুনির্দিষ্ট সিস্টেম প্রম্পট দেওয়া হবে: "এই ওয়েব কোডগুলোকে একটি নেটিভ Android Kotlin/Jetpack Compose প্রজেক্টে রূপান্তর করো, অথবা একটি অস্টিমাইজড Capacitor/WebView ইমপ্লিমেন্টেশন জেনারেট করো।"
- জেমিনাইয়ের রেসপন্স থেকে কোটলিন কোডগুলো এক্সট্র্যাক্ট করে base-app/src/main/java/com/example/app/MainActivity.kt এবং অন্যান্য ফাইলে প্রোগ্রামাটিকভাবে

ওভাররাইট করা হবে।

ধাপ ৫: লোকাল কম্পাইলেশন এবং এপিকে (APK) এক্সপোর্ট

এআই দ্বারা সফলভাবে কোড জেনারেট করে লোকাল ফাইলে সেভ করার পর, স্ক্রিপ্টটি স্বয়ংক্রিয়ভাবে কমান্ড-লাইন বিন্ড ট্রিগার করবে।

১. পাইথন স্ক্রিপ্টের শেষ অংশে সাবপ্রসেস (Subprocess) হিসেবে বিন্ড কমান্ড যুক্ত করতে হবে, অথবা ম্যানুয়ালি টার্মিনাল থেকে কমান্ড দেওয়া যায়:

```
cd base-app
gradle assembleRelease
```

বিকল্প হিসেবে, যদি প্রজেক্টটি গ্র্যাডল ছাড়া লাইটওয়েট হয়, তবে সরাসরি টারমিনালের ব্যাশ স্ক্রিপ্ট ব্যবহার করা যায়:

```
bash build.sh
```

২. বিন্ড প্রসেসটি কয়েকটি সুনির্দিষ্ট ধাপে সম্পন্ন হয়। প্রথমে aapt2 compile এবং link এর মাধ্যমে এক্সএমএল (XML) এবং রিসোর্স ফাইলগুলো প্রসেস হয়। এরপর javac দ্বারা জাভা বা কোটলিন ফাইলগুলো ক্লাস ফাইলে রূপান্তরিত হয়। এরপর d8 টুল এই ক্লাস ফাইলগুলোকে ডালভিক এক্সিকিউটেবল (DEX) ফাইলে কনভার্ট করে। পরিশেষে ফাইলগুলো একত্রে জিপ হয়ে zipalign দ্বারা মেমরি অপটিমাইজ হয় এবং apksigner দ্বারা ক্রিপ্টোগ্রাফিকভাবে সাইন হয়ে একটি চূড়ান্ত এপিকে ফাইলে পরিণত হয়। ৩. সফলভাবে বিন্ড সম্পন্ন হওয়ার পর আউটপুট ফাইলটি app-release.apk বা app-debug.apk হিসেবে ~/ai-app-engine/base-app/app/build/outputs/apk/release/ ডিরেক্টরিতে পাওয়া যাবে। ৪. এই ফাইলটিকে ফোনের ইন্টারনাল স্টোরেজে (যাতে সাধারণ ফাইল ম্যানেজার দিয়ে অ্যাক্সেস করা যায়) কপি করে নিতে কমান্ড দিতে হবে:

```
cp
app/build/outputs/apk[span_74](start_span)[span_74](end_span)[span_76]
(start_span)[span_76](end_span)[span_78](start_span)[span_78](end_span)
)/release/app-release.apk /sdcard/Download/
```

ধাপ ৬: লোকাল টেস্টিংয়ের জন্য AnyAPK সেটআপ

গুগল প্লে প্রোটেক্ট এবং অ্যান্ড্রয়েডের নতুন নিরাপত্তা রেস্ট্রিকশন অনুযায়ী অনেক সময় আনভেরিফায়েড ডেভেলপার সোর্স বা লোকাল ডিরেক্টরি থেকে সরাসরি এপিকে (APK) ইন্সটল সিস্টেম বাধা প্রদান করে। কোনো পিসির সাহায্য ছাড়া এই সাইডলোডেড অ্যাপগুলো নির্বিঘ্নে ইন্সটল করার জন্য **AnyAPK** নামের একটি ওপেন-সোর্স টুল ব্যবহার করা হবে, যা ওয়্যারলেস এডিবি (Wireless ADB) প্রোটোকল ব্যবহার করে কাজ করে এবং রুট পারমিশন দাবি করে না।

১. গিটহাব রিলিজ (GitHub Releases) পেজ থেকে AnyAPK-এর লেটেস্ট ভার্সন (anyapk.apk) ডাউনলোড করে ফোনে ইন্সটল করতে হবে। এটি কাজ করার জন্য ডিভাইসে অন্তত অ্যান্ড্রয়েড ১১ (Android 11) থাকা বাঞ্ছনীয়। ২.

ফোনের Settings > About Phone-এ গিয়ে 'Build Number'-এ ৭ বার ট্যাপ করে **Developer Options** চালু করতে হবে। ৩. Developer Options থেকে **Wireless Debugging** অপশনটি চালু করতে হবে। এজন্য ফোনটিকে অবশ্যই যেকোনো ওয়াইফাই নেটওয়ার্কের সাথে যুক্ত থাকতে হবে, যদিও অ্যাপটি ইন্সটল করার জন্য এক্সটার্নাল ইন্টারনেটের প্রয়োজন নেই, এটি সম্পূর্ণ লোকালহোস্টের মাধ্যমে কাজ করে। ৪. AnyAPK অ্যাপটি ওপেন করে "Enter Pairing Code" বা "Open Developer Settings"-এ ট্যাপ করতে হবে। স্ক্রিনটিকে স্প্লিট-স্ক্রিন (Split-screen) করে একপাশে AnyAPK এবং অন্যপাশে Wireless Debugging মেনু রাখতে হবে। ৫. Wireless Debugging মেনু থেকে "Pair device with pairing code" সিলেক্ট করলে একটি ৬-ডিজিটের কোড এবং একটি ডায়নামিক পোর্ট (Port) নম্বর দেখাবে। ৬. AnyAPK-তে ওই কোড এবং পোর্ট নম্বরটি সঠিকভাবে ইনপুট দিয়ে "Pair" বাটনে ট্যাপ করতে হবে। পেয়ারিং সম্পন্ন হলে স্ক্রিনে একটি সবুজ চেকমার্ক দেখা যাবে। ৭. পেয়ারিং সম্পন্ন হওয়ার পর, AnyAPK-এর ভেতরের ফাইল পিকার (File picker) দিয়ে /sdcard/Download/ ডিরেক্টরি থেকে নতুন তৈরি হওয়া app-release.apk সিলেক্ট করতে হবে। AnyAPK স্বয়ংক্রিয়ভাবে লোকাল এডিবি প্রোটোকল ব্যবহার করে অ্যাপটি সিস্টেমে ইন্সটল করে

দেবে, যা গুগল প্লে প্রোটেক্টর ভেরিফিকেশন স্কিপ করতে সক্ষম।

ফলব্যাক মেকানিজম: গিটহাব অ্যাকশনস (GitHub Actions) ভিত্তিক ক্লাউড সিআই/সিডি

লোকাল ডিভাইসে গ্র্যাডল রান করা অত্যন্ত সিপিইউ (CPU) এবং মেমরি-ইন্টেনসিভ কাজ। কম ক্ষমতার ডিভাইসে, বিশেষ করে যেগুলোতে র‍্যামের পরিমাণ কম, সেখানে টারমিনাল ব্লক করার সময় Process completed (signal 9) - press Enter এরর আসতে পারে। এটি নির্দেশ করে যে অ্যান্ড্রয়েড অপারেটিং সিস্টেম টারমিনালকে মেমরি স্বল্পতার কারণে ফোর্সভালি কিল (Kill) করে দিয়েছে। যদি লোকাল কম্পাইলেশন ফেইল করে, তবে এআই ইঞ্জিনটিকে ফলব্যাক হিসেবে গিটহাব অ্যাকশনস (GitHub Actions)-এর সাথে সংযুক্ত করা যেতে পারে।

এই পদ্ধতিতে ইঞ্জিনটি জেমিনাই দিয়ে কোড জেনারেট করার পর লোকাল গ্র্যাডল রান না করে, সরাসরি গিটহাব এপিআই (GitHub API) ব্যবহার করে একটি প্রাইভেট রিপোজিটরিতে কোড এবং ডিরেক্টরি পুশ করবে। গিটহাব রিপোজিটরিতে আগে থেকে কনফিগার করা একটি .github/workflows/build.yml ফাইল থাকবে। কোড পুশ হওয়ামাত্রই গিটহাবের উবুন্টু (Ubuntu) রানারে গ্র্যাডল ব্লক ট্রিগার হবে। কম্পাইলেশন শেষে ক্লাউডেই এপিকে এবং এএবি (AAB) ফাইল জেনারেট হবে, যা আর্টিফ্যাক্ট (Artifact) বা রিলিজ হিসেবে সংযুক্ত হবে। এরপর মোবাইল ব্রাউজার থেকে সরাসরি সেই এপিকে ফাইলটি ডাউনলোড করে AnyAPK দিয়ে ইন্সটল করা সম্ভব। এই পদ্ধতিটি ডিভাইসের প্রসেসিং পাওয়ারের সীমাবদ্ধতা পুরোপুরি দূর করে এবং একইসাথে কোনো পিসি বা ল্যাপটপের প্রয়োজনীয়তা এড়িয়ে সম্পূর্ণ মোবাইলের মাধ্যমেই অ্যাপ রিলিজের উদ্দেশ্য সফল করে।

উপসংহার

একটি ল্যাপটপ বা ডেস্কটপের ব্যবহার সম্পূর্ণ এড়িয়ে শুধুমাত্র অ্যান্ড্রয়েড মোবাইল ডিভাইস, টারমিনাল ইকোসিস্টেম এবং গুগল এআই স্টুডিওর (জেমিনাই এপিআই) সাহায্যে একটি এপিকে জেনারেটর ইঞ্জিন তৈরি করা প্রযুক্তিগতভাবে সম্পূর্ণরূপে সম্ভব। গুগল এআই স্টুডিওর নিজস্ব ইউজার ইন্টারফেস সরাসরি এপিকে ডাউনলোডের সুবিধা না দিলেও, জেমিনাই এপিআইয়ের অত্যন্ত শক্তিশালী কোড জেনারেশন সক্ষমতাকে টারমিনালের লোকাল ব্লক এনভায়রনমেন্ট (OpenJDK, SDK, Gradle)-এর সাথে পাইথন স্ক্রিপ্টের মাধ্যমে সংযুক্ত করে একটি অটোমেটেড পাইপলাইন তৈরি করা যায়। প্রসেসিংয়ের সময় aapt2 আর্কিটেকচার সংক্রান্ত জটিলতাগুলো সঠিক প্রোপারটিজ ওভাররাইড করার মাধ্যমে সমাধান করা যায়। পরিশেষে, AnyAPK-এর ওয়্যারলেস এডিটিং ইমপ্লিমেন্টেশন নিশ্চিত করে যে, জেনারেট করা অ্যাপ্লিকেশনটি কোনো কর্পোরেট ডেভেলপার ভেরিফিকেশন বা ডেডিকেটেড কম্পিউটারের সাহায্য ছাড়াই সরাসরি ফোনে ইন্সটল ও পরীক্ষা করা যাবে। এই মেটা-আর্কিটেকচারটি এলনাইন মেটা-ডিজাইনার (L9_META_DESIGNER) ফিলোসফির সাথে সামঞ্জস্যপূর্ণ, যা মোবাইল-ফার্স্ট সফটওয়্যার ডেভেলপমেন্ট এবং জেনারেটিভ এআই-এর একত্রীকরণের ক্ষেত্রে একটি বাস্তবসম্মত ও বৈপ্লবিক রূপরেখা প্রদান করে।

যেসব কাজের উদ্ধৃতি যোগ করা হয়েছে

1. Google AI Studio Build Tab: How to Go from Idea to Deployed App in Minutes | MindStudio, <https://www.mindstudio.ai/blog/google-ai-studio-build-tab-guide>
2. Google AI Studio vs Bolt vs Lovable: Which AI App Builder Should You Use? | MindStudio, <https://www.mindstudio.ai/blog/google-ai-studio-vs-bolt-vs-lovable>
3. Build an App with AI: Get a Shareable URL | Google Cloud, <https://cloud.google.com/use-cases/how-to-build-an-app-with-ai>
4. Bring any idea to life: Google AI Studio at I/O 2026, <https://blog.google/innovation-and-ai/technology/developers-tools/google-ai-studio-io-2026/>
5. Build Android Apps in Google AI Studio | Gemini API, <https://ai.google.dev/gemini-api/docs/aistudio-android>
6. Building and Publishing a Complete Full-Stack Web and Native Android App on Google AI Studio - DEV Community, <https://dev.to/sreeraj-sreenivasan/building-and-publishing-a-complete-full-stack-web-and-native->

android-app-on-google-ai-studio-14h0 7. Build apps in Google AI Studio | Gemini API, <https://ai.google.dev/gemini-api/docs/aistudio-build-mode> 8. Add Cloud Firestore and Authentication to your Google AI Studio app | Develop with AI assistance, <https://firebase.google.com/docs/ai-assistance/ai-studio-integration> 9. Build an app with the Gemini API | Firebase Studio - Google, <https://firebase.google.com/docs/studio/build-gemini-api-app> 10. Backend Environment Error: Missing Java JDK / Gradle Toolchain in AI Studio Container Blocks Compilation - Google AI Developers Forum, <https://discuss.ai.google.dev/t/backend-environment-error-missing-java-jdk-gradle-toolchain-in-a-i-studio-container-blocks-compilation/171101> 11. How to use Google AI Studio to quickly build (or “vibe code”) and deploy apps - Appinventiv, <https://appinventiv.com/blog/building-apps-with-ai-studio/> 12. APK maker: Easy to Use No Coding Required - Andromo, <https://www.andromo.com/app-builder/apk-maker/> 13. Build Android Apps Online | No Android Studio Required - RapidNative, <https://www.rapidnative.com/android-app-builder-online> 14. APK Builder Online | Free APK Generator for Android - Fastshot, <https://fastshot.ai/apk-builder> 15. AppsGeyser: No code App Builder | Create an App, <https://appsgeyser.com/> 16. Appilix — Convert Any Website into a Mobile App, <https://appilix.com/> 17. Median.co: Convert any website to iOS & Android native apps, <https://median.co/> 18. HTML & Web to APK: AppMint - Apps on Google Play, <https://play.google.com/store/apps/details?id=com.freewebtoapk> 19. Building an AI-Powered Web App with Google's Gemini API in JavaScript, <https://www.cygner.net/blog/building-an-ai-powered-web-app-with-google-gemini-api-in-javascript> 20. GitHub - ionic-team/capacitor: Build cross-platform Native Progressive Web Apps for iOS, Android, and the Web ⚡, <https://github.com/ionic-team/capacitor> 21. Using Android CLI Tools · ionic-team capacitor · Discussion #3224 · GitHub, <https://github.com/ionic-team/capacitor/discussions/3224> 22. [android] termux support - Feature requests - Obsidian Forum, <https://forum.obsidian.md/t/android-termux-support/49711> 23. Create advanced data driven Gemini API apps - YouTube, https://www.youtube.com/watch?v=_Op9QfXDBuM 24. No Laptop? No Problem: Setting Up a Professional Kotlin & Java Environment in Termux, <https://dev.to/brighto7700/no-laptop-no-problem-setting-up-a-professional-kotlin-java-environment-in-termux-4hbg> 25. Building Android applications in Termux using Gradle - Grokipedia, https://grokipedia.com/page/Building_Android_applications_in_Termux_using_Gradle 26. azmaveth/termux-android-hello - GitHub, <https://github.com/azmaveth/termux-android-hello> 27. Build your app from the command line | Android Studio - Android Developers, <https://developer.android.com/build/building-cmdline> 28. Build Android APKs in Termux | PDF - Scribd, <https://www.scribd.com/document/978196487/Easily-Build-Android-APKs-on-Device-in-Termux-DocsBuildAPKs> 29. Getting started with the Gemini API and Web apps - Google for Developers, <https://developers.google.com/learn/pathways/solution-ai-gemini-getting-started-web> 30. Building Android Applications with Gradle (Kotlin) : r/termux - Reddit, https://www.reddit.com/r/termux/comments/1ik09pq/building_android_applications_with_gradle_kotlin/ 31. Willie169/termux-android-sdk-ndk - GitHub, <https://github.com/Willie169/termux-android-sdk-ndk> 32. illegalvoid/termux-build-apps: build APKs with Termux, without needing Proot - GitHub, <https://github.com/illegalvoid/termux-build-apps> 33. Build android app on termux - Gist - GitHub, https://gist.github.com/tmatz/817bf03433e059bf89c63dc33f286ccb?permalink_comment_id=6209999 34. How to Make/Build Android .apk in Termux #25793 - GitHub,

<https://github.com/termux/termux-packages/discussions/25793> 35. Gradle is a freaking pain to work with. Please help. : r/termux - Reddit, https://www.reddit.com/r/termux/comments/1tzbb62/gradle_is_a_freaking_pain_to_work_with_please_help/ 36. Gradle ile uğraşmak tam bir baş belası. Lütfen yardım edin. : r/termux - Reddit, https://www.reddit.com/r/termux/comments/1tzbb62/gradle_is_a_freaking_pain_to_work_with_please_help/?tl=tr 37. reapercanuk39/termux-kotlin-app: Termux Android Terminal Emulator - Fully converted to Kotlin! A modern, type-safe fork of the official termux-app with 100% Kotlin codebase. - GitHub, <https://github.com/reapercanuk39/termux-kotlin-app> 38. Automating Android APK & AAB Generation with GitHub Actions - Medium, <https://medium.com/@myofficework000/automating-android-apk-aab-generation-with-github-actions-0910888de1b1> 39. anyapk for Android - Download the APK from Uptodown, <https://anyapk.en.uptodown.com/android> 40. ANYAPK. Bypass Google And Install Any APK You Want On The Device You Own. - Reddit, https://www.reddit.com/r/degoogle/comments/1rintnn/anyapk_bypass_google_and_install_any_apk_you_want/ 41. GitHub - sam1am/anyapk: Install any apk on the device you own., <https://github.com/sam1am/anyapk> 42. anyapk/USAGE.md at main - GitHub, <https://github.com/sam1am/anyapk/blob/main/USAGE.md> 43. I stopped waiting for Android Studio and built my first app in Google AI Studio instead, <https://www.makeuseof.com/i-built-my-first-android-app-in-google-ai-studio/> 44. Wireless debug activation · Issue #42 · sam1am/anyapk - GitHub, <https://github.com/sam1am/anyapk/issues/42> 45. Create Commit By GitHub API · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/create-commit-by-github-api> 46. Actions · MasonAmerica/webview-apk · GitHub, <https://github.com/MasonAmerica/webview-apk/actions> 47. Build apk automatically with Github Action | by Anang Pambudi - Medium, <https://medium.com/@anangpambudi101/build-apk-automatically-with-github-action-49373c25d9d0> 48. automated-build-android-app-with-github-action - GitHub Marketplace, <https://github.com/marketplace/actions/automated-build-android-app-with-github-action> 49. I built a free GitHub Actions workflow generator for mobile apps (iOS & Android) - Reddit, https://www.reddit.com/r/github/comments/1r7l270/i_built_a_free_github_actions_workflow_generator/